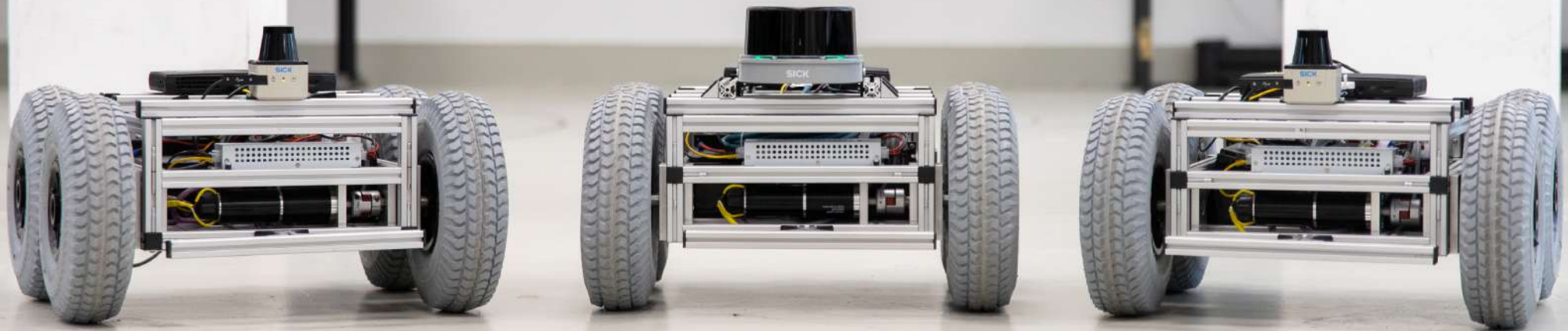


Robotik

Prof. Dr. Thomas Wiemann - FB AI - Robotik



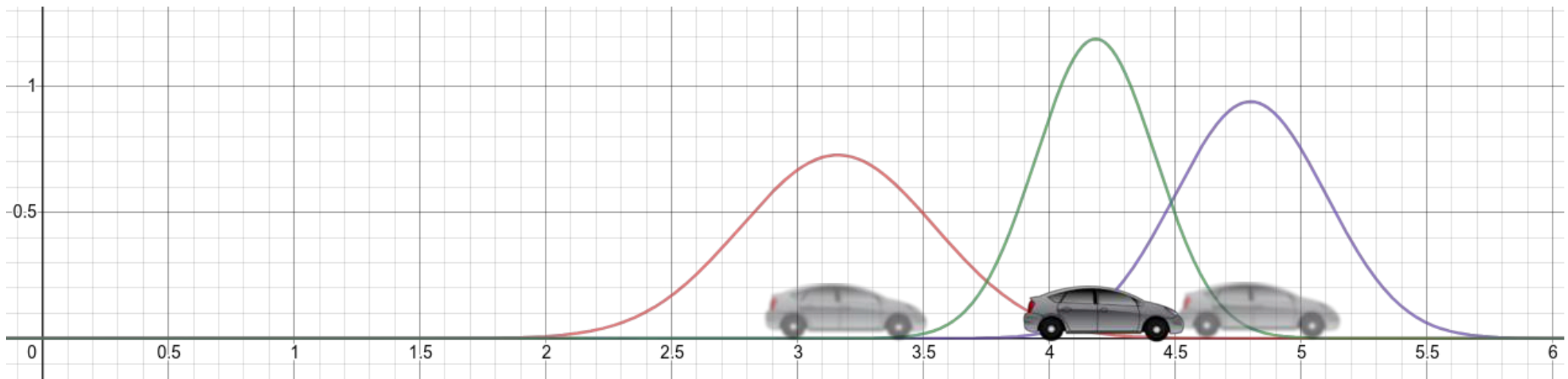
Hochschule Fulda
University of Applied Sciences





Kalman-Filter - Die Motivation

- ▶ Einsatz mehrerer Sensoren, deren Messwerte mit Unsicherheiten belegt sind
- ▶ Ziel: Zustandsschätzung basierend auf mehreren unabhängigen Messungen, so dass
 - ▶ ... die Gesamtunsicherheit reduziert wird
 - ▶ ... glaubwürdige Informationen stärker gewichtet werden als unplausible
 - ▶ ... die abgeleitete Lösung statistisch "besser" ist als die Einzellösungen



Modellierung aller Schätzungen durch Gaußfunktionen!



Kalman-Filter als spezieller Bayes-Filter

$$\mathbf{P}(\mathbf{x}_{t+1} \mid \mathbf{x}_t, \mathbf{u}_t) = \mathcal{N}(\mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t, \Sigma_{\mathbf{u}})(\mathbf{x}_{t+1})$$

$$\mathbf{P}(\mathbf{z}_t \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{H}\mathbf{x}_t, \Sigma_{\mathbf{z}})(\mathbf{z}_t)$$

n	Dimension Zustand
m	Dimension Aktion
l	Dimension Messung

- $\mathbb{1}$ Einheitsmatrix ($n \times n$)-dimensional
- $\mathbf{A}$ Transitionsmodell ($n \times n$)-dimensional, modelliert spontane Transitionen. Ist gleich $\mathbb{1}$, wenn keine spontanen Transitionen erfolgen
- $\mathbf{B}$ Aktionsmodell ($n \times m$)-dimensional, konvertiert $\mathbf{u}$ in den Zustandsraum
- $\Sigma_{\mathbf{u}}$ Kovarianzmatrix für Aktionsmodell ($n \times n$)-dimensional
- $\mathbf{H}$ Sensormodell, ($l \times n$)-dimensional. Konvertiert Zustand in den Messraum
- $\Sigma_{\mathbf{z}}$ Kovarianzmatrix für Sensormodell ($l \times l$)-dimensional





1. Prädiktion:

- Sage resultierenden Zustand für Aktion/Ereignis voraus
- Schätze den Fehler nach aktuellem Fehlermodell ab
- Führe die Aktion aus (bzw. unabhängiges Ereignis geschieht)

2. Filterung:

- Miss resultierenden Zustand
- Ggf. mach mehrere unabhängige Messungen (die sich widersprechen dürfen)
- Nimm als aktuellen Zustand den an, der maximal gut zu allen Messungen und dem letzten bekannten Fehlermodell passt
- Aktualisiere Fehlermodell (!)
- Weiter bei 1.



A-priori-Vorhersagen für $t + 1$:

$$\begin{aligned}\underline{x}_{t+1} &:= \mathbf{A}\mu_t + \mathbf{B}u \\ \underline{\Sigma}_{t+1} &:= \mathbf{A}\Sigma_t\mathbf{A}^T + \Sigma_u\end{aligned}$$

← Vorhergesagter Zustand zu $t + 1$ nach u

← Vorhergesagte Kovarianz zu $t + 1$ nach u

Filterung / Korrektur:

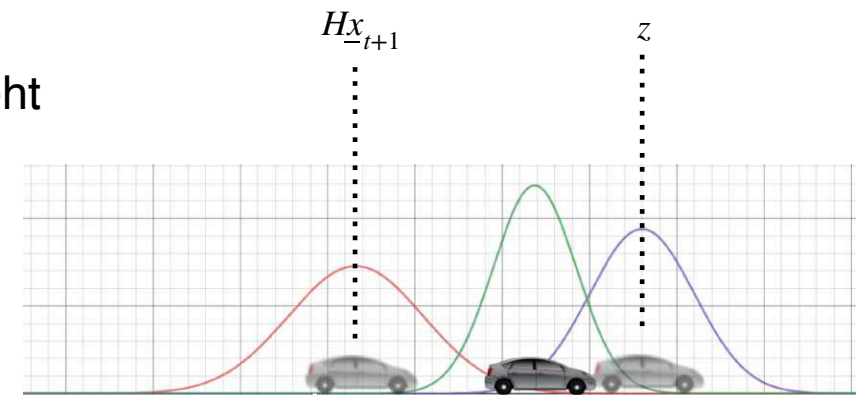
$$\begin{aligned}\mu_{t+1} &= \underline{x}_{t+1} + \mathbf{K}_{t+1}(\underline{z} - \mathbf{H}\underline{x}_{t+1}) \\ \Sigma_{t+1} &= (\mathbf{I} - \mathbf{K}_{t+1}\mathbf{H})\underline{\Sigma}_{t+1}\end{aligned}$$

Rote Kurve

grüne Kurve

$\mathbf{K}_{t+1}$ sagt, wie hoch der Vorhersagefehler in neuen Wert eingeht
"Kalman Gewinnmatrix (*Kalman Gain*)":

$$\mathbf{K}_{t+1} = \underline{\Sigma}_{t+1}\mathbf{H}^T(\mathbf{H}\underline{\Sigma}_{t+1}\mathbf{H}^T + \Sigma_z)^{-1}$$





Eingabe : x_t aktueller Zustand, Σ_t aktuelles Fehlermodell, sowie ein Evidenz-Paar (u, z)

Ausgabe: Die aktualisierten x_{t+1}, Σ_{t+1}

1: *Prädiktion mit Hilfe der Aktion u*

2: $\bar{x}_{t+1} = Ax_t + Bu$

3: $\bar{\Sigma}_{t+1} = A\Sigma_t A^T + \Sigma_u$

4: *Korrektur mit Hilfe der Messung z*

5: $K_{t+1} = \bar{\Sigma}_{t+1} H^T (H \bar{\Sigma}_{t+1} H^T + \Sigma_z)^{-1}$

6: $x_{t+1} = \bar{x}_{t+1} + K_{t+1}(z - H\bar{x}_{t+1})$

// Zustand $x_i \hat{=}$ Mittelwert μ_i

7: $\Sigma_{t+1} = (\mathbb{1} - K_{t+1}H) \bar{\Sigma}_{t+1}$

8: **return** x_{t+1}, Σ_{t+1}



- ▶ Gauß / Normalverteilung nicht zwingend für Kalman-Filter:
 - ▶ Systemrauschen mit Mittelwert 0
 - ▶ Fehlerquellen untereinander mit Kovarianz 0
 - ▶ Lineare Transitionen
 - ▶ Lineare Abhängigkeiten zwischen Zuständen und Sensormessungen
- ➔ **Unter diesen Bedingungen ist der Kalman-Filter optimal!**
 - ▶ Kein anderes Verfahren schätzt die Systemzustände mit geringeren erwarteten Abständen zwischen geschätztem und tatsächlichem Zustand
 - ▶ Kalman-Filter nutzt die vorhandenen Informationen optimal aus



Beispiel: Gleichförmige Bewegung + GPS

Erinnerung:

$$\underline{x}_{t+1} := A\mu_t + Bu$$

$$\underline{\Sigma}_{t+1} := A\underline{\Sigma}A^T + \underline{\Sigma}_u$$

$$\mu_{t+1} = \underline{x}_{t+1} + \mathbf{K}_{t+1}(z - \mathbf{H}\underline{x}_{t+1})$$

$$\underline{\Sigma}_{t+1} = (1 - \mathbf{K}_{t+1}\mathbf{H})\underline{\Sigma}_{t+1}$$

$$\mathbf{K}_{t+1} = \underline{\Sigma}_{t+1}\mathbf{H}^T(\mathbf{H}\underline{\Sigma}_{t+1}\mathbf{H}^T + \underline{\Sigma}_z)^{-1}$$

Start bei $x_0 = 0$, $\sigma_0 = 0$, $\Sigma_0 = [0]$, sei $u = \Delta x = 3,2$

Sei $\sigma_u = (|\Delta x|)/10 = 0,32$

$$\Rightarrow \Sigma_u = [\sigma_u^2] = [0,32^2]$$

$$\Rightarrow \underline{x}_1 = 3,2$$

$$\Rightarrow \underline{\Sigma}_1 = \Sigma_0 + \Sigma_u = [0,32^2]$$

Weiter gelten $\mathbf{H} = [1]$ und $\Sigma_z = [\sigma_z^2] = [0,05^2]$

$$\Rightarrow \mathbf{K}_1 = (0,32^2)/(0,32^2 + 0,05^2) = 0,97616$$

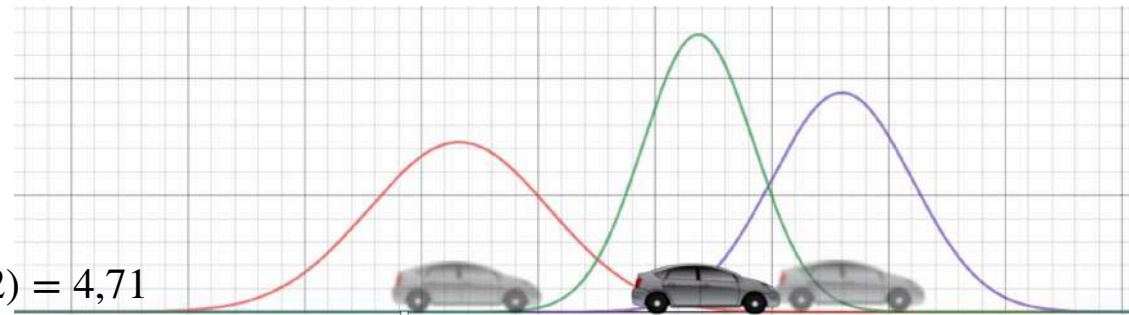
$$\Rightarrow \underline{\Sigma}_1 = (1 - \mathbf{K}_1)\underline{\Sigma}_1 = [0,00244^2]$$

Sei Messwert $z_1 = 4,75$

$$\Rightarrow \mu_1 = \underline{x}_1 + \mathbf{K}_1(z_1 - \mathbf{H}\underline{x}_1) = 3,2 + 0,97616(4,75 - 3,2) = 4,71$$

Messung gewinnt, da sie deutlich zuverlässiger ist:

$$\sigma_z \ll \sigma_u$$





Für einen hypothetischen mobilen Roboter bestehe der Zustandsraum aus folgenden Komponenten:

- den Koordinaten x, z
- der Orientierung θ
- dem Luftdruck in Osnabrück λ
- dem Namen des Papstes π

Unabhängig von den Aktionen des Roboters kann sich der Luftdruck von einem zum nächsten Zustand von selbst mit einer Standardabweichung von 0.001 hPa ändern. Die Roboteraktionen ändern weder den Luftdruck, noch den Namen des Papstes. Der Roboter kann die aktuellen Werte von x, z, θ und λ messen.

- (1) Geben Sie die Matrizen **A**, **B** und **H** für die beschriebene Domäne an!
- (2) Modellieren Sie die Aktionen *Rotation* und *Translation* mit gegebenen Standardabweichungen!
- (3) Beispielrechnung von drei Iterationen für gegebene Initialwerte und Aktionen



Beispielaufgabe - Diskussion

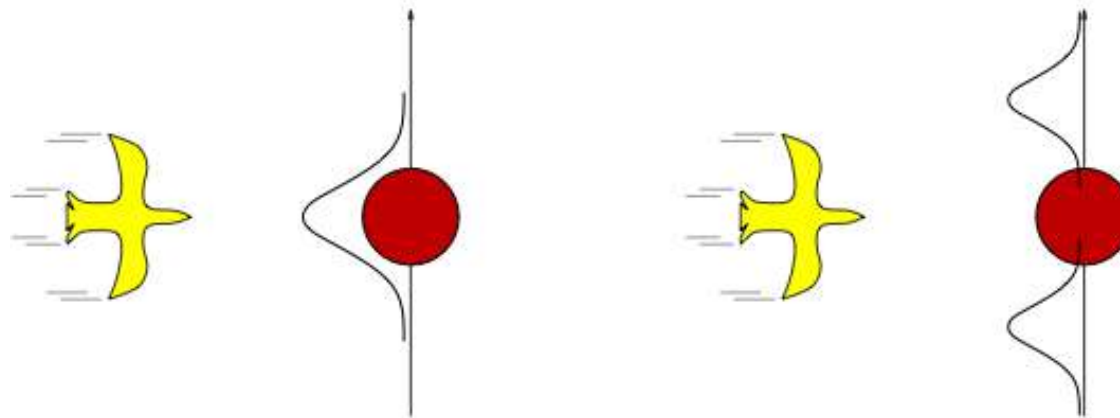
- Besprechung Form und Dimension der erforderlichen Matrizen

Lösung: Sei $x = \begin{pmatrix} x \\ z \\ \theta \\ \lambda \\ \pi \end{pmatrix}$ der Zustandsraum, $u = \begin{pmatrix} \Delta x \\ \Delta z \\ \Delta \theta \end{pmatrix}$ der Aktionsraum und $Z = \begin{pmatrix} x \\ z \\ \theta \\ \lambda \end{pmatrix}$ der Messraum



Grenzen des Kalmanfilters

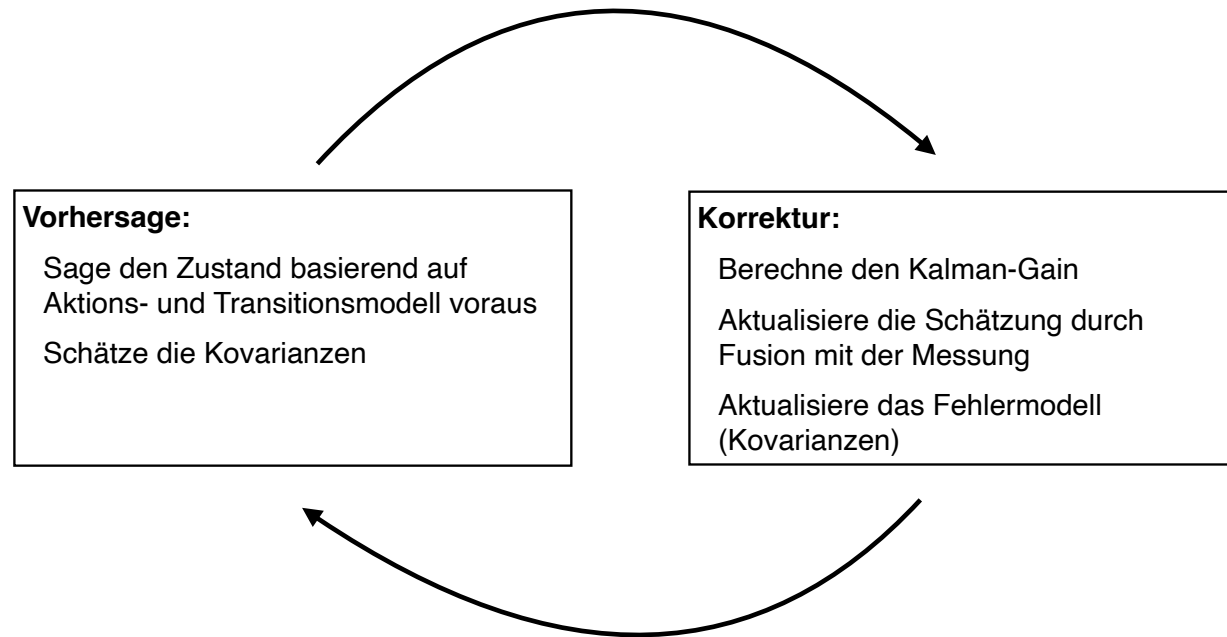
- ▶ Kalman-Filter praktisch auch angewandt, wenn Optimalitäts-voraussetzung nicht gegeben (z.B. korrelierte Fehlerquellen)
- ▶ Unimodale Verteilungen modellieren manche Aktionen nicht
- ▶ ➡ später arbeiten wir oft mit multimodalen Verteilungen (allg. Bayes-F.)



- ▶ In der Robotik sind Transitionen meist/oft nicht linear
- ▶ ➡ Erweiterter Kalman-Filter (EKF): Approximative Linearisierung eines nichtlinearen Transitions- und/oder Sensormodells (Jakobi-Matrizen d. nichtlinearen Funktionen)



Zusammenfassung



- Berechnung einer Zustandsschätzung, die die Informationen über den Systemzustand und durch Messdaten gewonnen Erkenntnisse optimal ausnutzt
- Unimodales Verfahren, man bekommt immer nur den wahrscheinlichsten Zustand
- Multimodale-Verfahren kommen dann später zur Lokalisierung!



Gliederung

Lokalisierung

1. Grundlagen
2. Bayes- und Kalmanfilter
- 3. Inkrementelle Lokalisierung**
4. Lokalisierung in Karten
5. Navigation
6. Ausblick

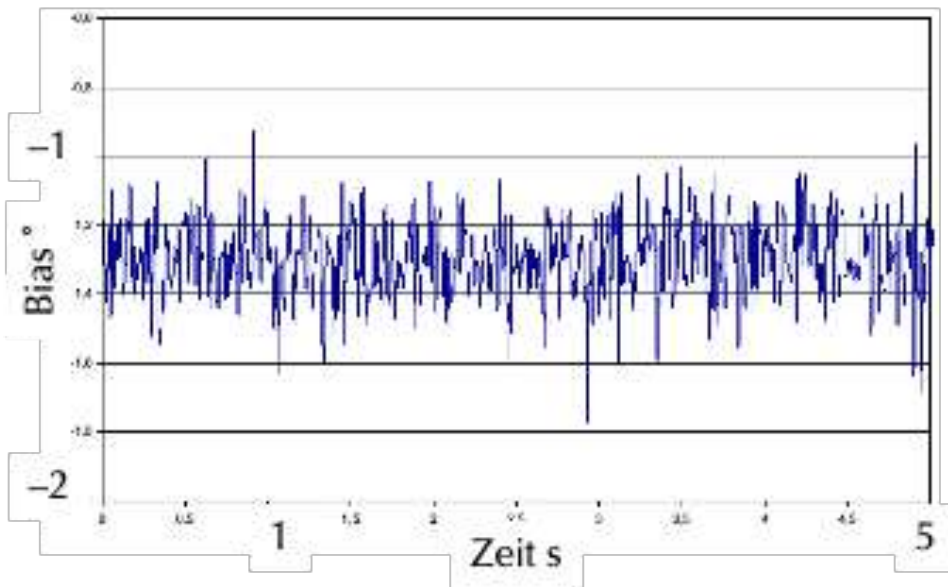


- ▶ Praktisch alle mobilen Roboter nutzen Odometriedaten („fast geschenkt!“)
- ▶ Normalerweise fusioniert mit anderen Messwerten
- ▶ Fusioniere mit solchen Messwerten, die unabhängig sind und anderen systematischen Fehlern unterliegen
- ▶ Fusionierung kann auf unterschiedliche Arten gehen (Kalman-Filter ist eine davon)
- ▶ In diesem Abschnitt Fusion von Odometriedaten mit
 1. Gyro-Daten („Gyrodometrie“)
 2. Laser-Winkelhistogrammen

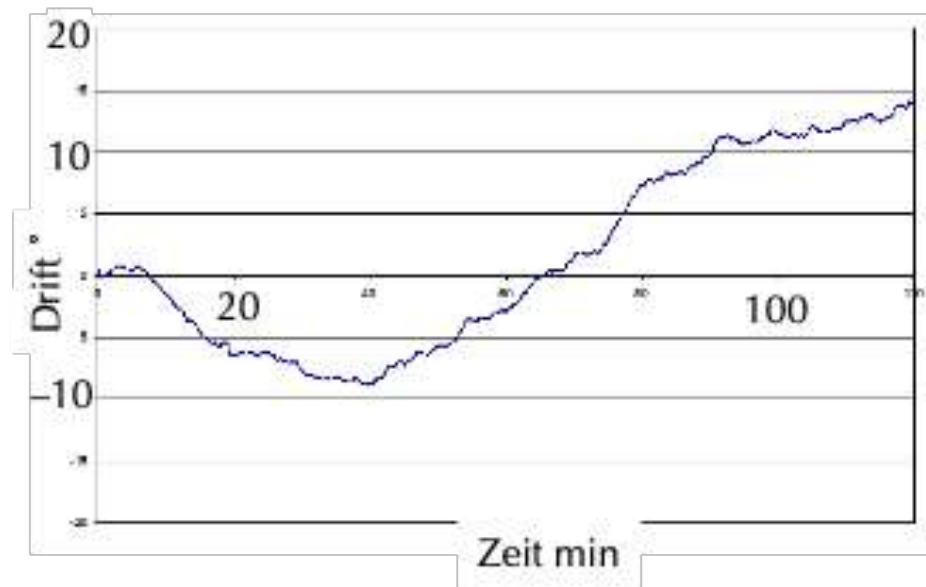


Wdh.: Gyrofehler - Qualitativ

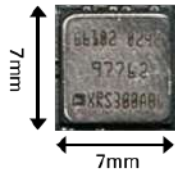
- Bias-Rauschen des ADXRS300 (Messwerte bei still stehendem Sensor)
- Bei Fahrt zusätzlich Salt & PepperNoise!



Lokal: Gauß-Rauschen
($\mu \approx -1.3^\circ$, $\sigma \approx 0.1^\circ$)

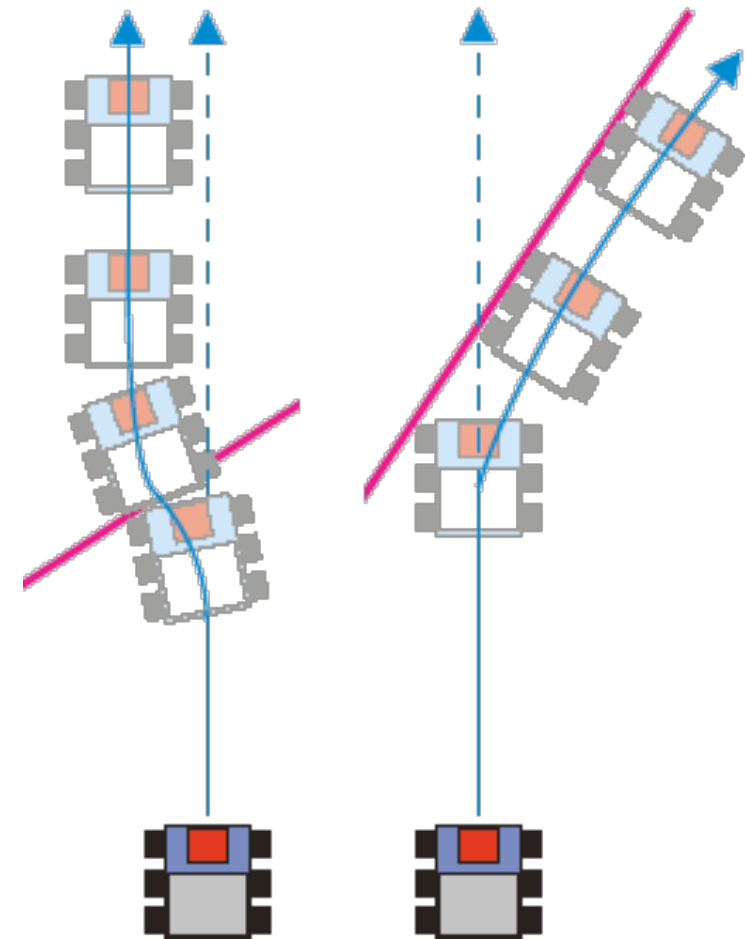


Global:





- ▶ Gyros unterliegen systematischer Drift
- ▶ Kurzfristig sind sie verlässlich, besonders auch bei Drehung
- ▶ Odometrie erzeugt kumulative Messfehler
- ▶ Die größten Messfehler entstehen an „kritischen“ Stellen:
 - Drehungen, „Holpern“, Schlupf
- ▶ Idee: Fusioniere Odo und Gyro
 - Variante 1: Bei Differenz nimm den aktuell „vermutlich verlässlicheren“ Wert
 - Variante 2: Fusioniere Odometrie und Gyro durch Kalman-Filter



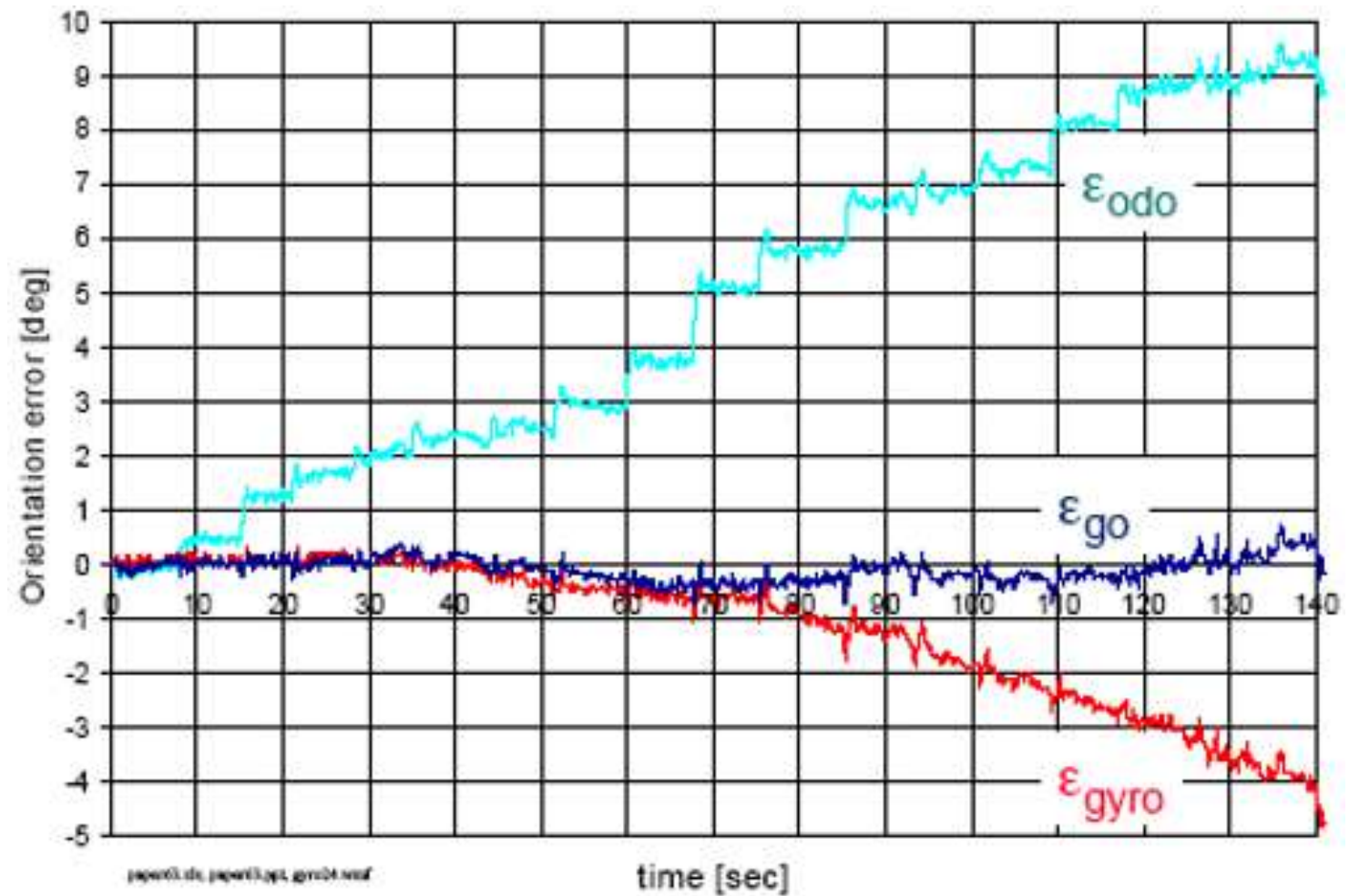


Wdh.: Deterministische Korrektur

if $\left(\left| \Delta\theta_{\text{gyro},i} - \Delta\theta_{\text{odo},i} \right| > \Delta\theta_{\text{min}} \right)$

then $\theta_i := \theta_{i-1} + t\Delta\theta_{\text{gyro},i}$

else $\theta_i := \theta_{i-1} + t\Delta\theta_{\text{odo},i}$





- ▶ ...eigentlich komplette Pose (3D) und Gyro 2D (Messwert und Bias) in EKF
- ▶ Nichtlinearität wegen Gyro-Integration [Solda et al. IAV-2004]
- ▶ Hier vereinfacht:
 - Eindimensional: Beachte lediglich Orientierung nach Gyro (linearer Messwert)
 - Fusion von Odometrie (lineare $\Delta\theta$ -Schätzung aufgrund erfolgter Radumdrehung) mit Gyro-Messwert
 - Bezug auf feste Zeitintervalle Δt , z.B. 1s
- ▶ Entspricht strukturell dem Beispiel Auto mit GPS aus dem vorherigen Abschnitt



- ▶ Orientierungsschätzung Odometrie:
 - Betrachtung in festen Intervallen von 1s
 - „Aktion“ $u = \Delta\theta = (v_L - v_R)/b$: Drehung innerhalb 1s
 - Transitionsmodell: $\mathbf{A} = \mathbf{B} = \mathbf{1}$
 - Varianz wäre empirisch zu ermitteln, hier „geraten“:
 - Transitions-Kovarianzmatrix:
 - $\Sigma_u = [\sigma_u^2]$, wobei $\sigma_u = (|\Delta\theta| + c)/10$ für festen Transitionsfehleranteil (Drehwinkelrauschen) c
- ▶ Orientierungsschätzung Gyro (empirisch ermittelt):
 - Feste Messfehler (z.B. Konstanter Offset von -1.3°) werden außerhalb des Filters behandelt
 - Standardabweichung $\sigma_z = 0.1^\circ = 0.00175 \text{ rad}$
 - Sensormodell: $\mathbf{H} = \mathbf{1}$
 - Sensor-Kovarianzmatrix: $\Sigma_z = [\sigma_z^2] = [0.00175^2]$



Gyrodometrie mit Kalmanfilter (2)

Erinnerung: $\underline{x}_{t+1} := \mathbf{A}\underline{\mu}_t + \mathbf{B}u$; $\underline{\Sigma}_{t+1} = \mathbf{A}\underline{\Sigma}_t\mathbf{A}^\top + \underline{\Sigma}_u$ $\underline{\mu}_{t+1} = \underline{x}_{t+1} + \mathbf{K}_{t+1}(\mathbf{z} - \mathbf{H}\underline{x}_{t+1})$
 $\mathbf{K}_{t+1} = \underline{\Sigma}_{t+1}\mathbf{H}^\top(\mathbf{H}\underline{\Sigma}_{t+1}\mathbf{H}^\top + \underline{\Sigma}_z)^{-1}$ $\underline{\Sigma}_{t+1} = (\mathbf{1} - \mathbf{K}_{t+1}\mathbf{H})\underline{\Sigma}_{t+1}$

Start bei (0,0,0), $\sigma_0 = 0$, $\Sigma_0 = [0]$, sei $u = \Delta\theta = 0.2$

Transitionsfehler $c := 0.1$, also $\sigma_u = (|\Delta\theta| + c)/10 = 0.03$

→ $\Sigma_u = [\sigma_u^2] = [0.03^2]$,

→ $\underline{x}_1 = [0.2]$, $\underline{\Sigma}_1 = \Sigma_0 + \Sigma_u = [0.03^2]$

Weiter gelten $\mathbf{H} = [1]$ und $\Sigma_z = [\sigma_z^2] = [0.00175^2]$ (s.o.), also

→ $\mathbf{K}_1 = [(0.03^2) / (0.03^2 + 0.00175^2)] = [0.99966]$

→ $\underline{\Sigma}_1 = (\mathbf{1} - \mathbf{K})\underline{\Sigma}_1 = [0.0005532^2]$

Sei Messwert $z_1 = 0.19$

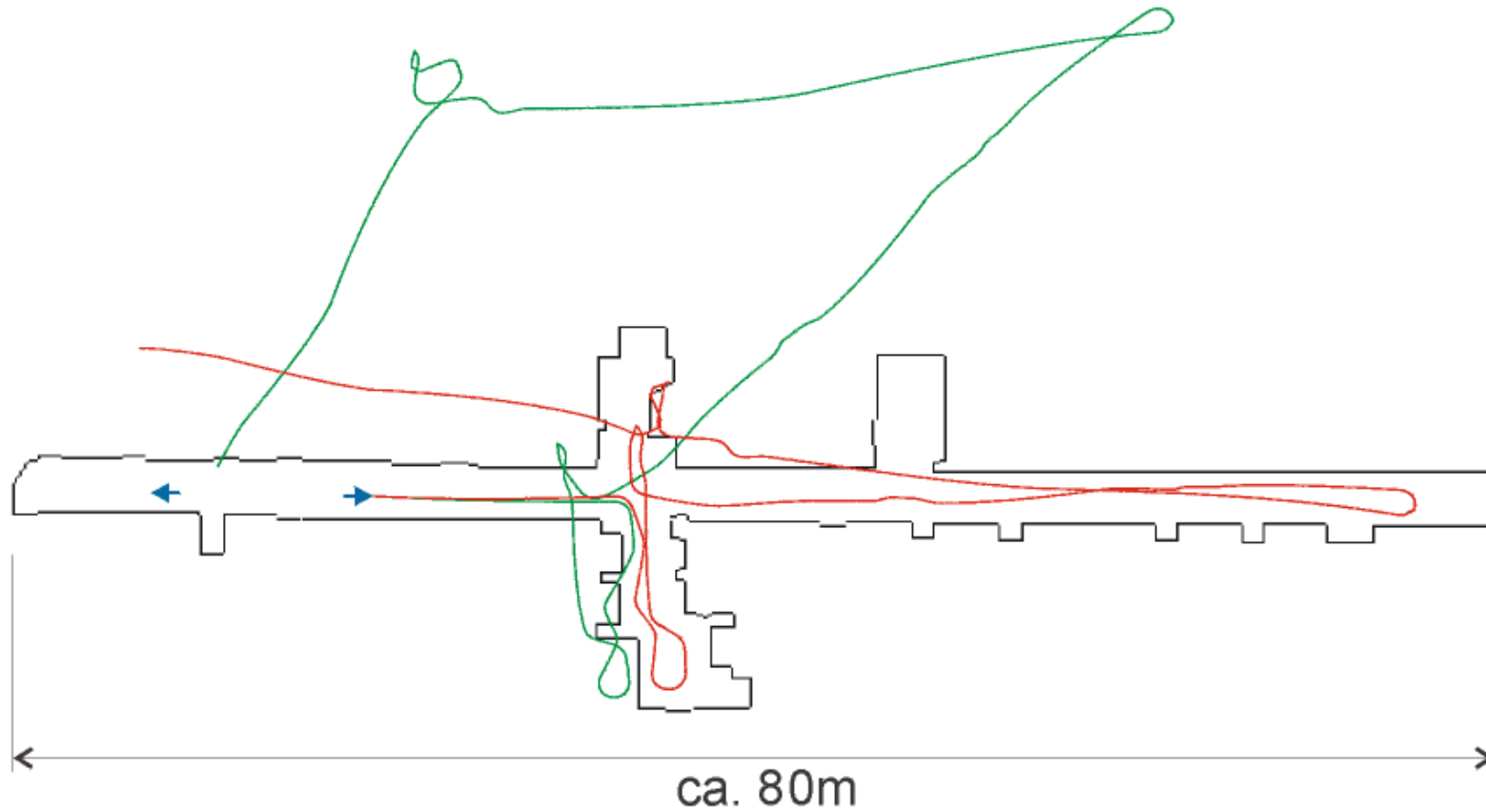
→ $\underline{\mu}_1 = \underline{x}_1 + \mathbf{K}(z_1 - \mathbf{H}\underline{x}_1) = [0.2] + [0.99966(0.19 - 0.2)] = [0.1900034]$

(Messung „gewinnt“, da sie deutlich zuverlässiger ist: $\sigma_z \ll \sigma_u$)



Beispiel: Odometrie + KF

- Positionsschätzung mittels Odometrie
- Positionsschätzung mittels Fusion von Odometrie und Gyroskop





Taylorapproximation

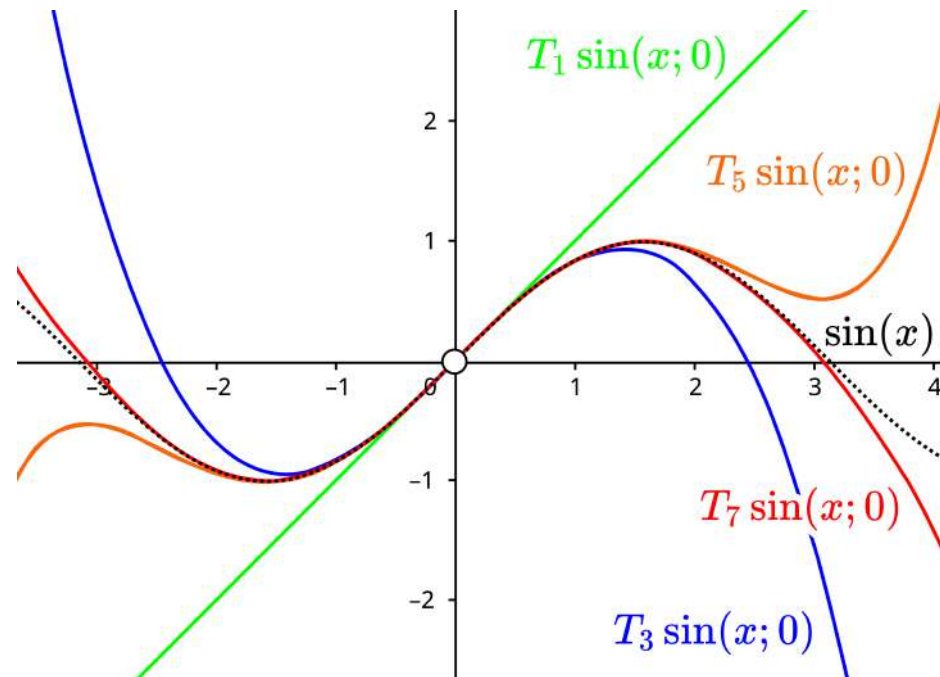
- Berechne eine beliebige Funktion f approximativ um einen Entwicklungspunkt a

$$\sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$$

$n!$ = Fakultät von n

a = reelle oder komplexe Zahl

$f^{(n)}(a)$ = n -te Ableitung von f , betrachtet an Punkt a



[https://de.wikipedia.org/wiki/Taylorreihe#/media/Datei:Taylor_Approximation_of_sin\(x\).svg](https://de.wikipedia.org/wiki/Taylorreihe#/media/Datei:Taylor_Approximation_of_sin(x).svg)



- Bring nicht-lineare Aktions- und Sensormodelle f und h in das Kalman-Filter-Verfahren ein

Ersetze Zustandsvorhersage $\underline{\mathbf{x}}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}$ durch:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u})$$

Ersetze Messvorhersage (Matrix $\mathbf{H}$) durch $h(\underline{\mathbf{x}})$, also die Zustandsaktualisierung, durch

$$\mathbf{x}_{t+1} = \underline{\mathbf{x}}_{t+1} + \mathbf{K}_{t+1}(\mathbf{z}_{t+1} - h(\underline{\mathbf{x}}_{t+1}))$$

- ⇒ Ersetze die Matrizen $\mathbf{A}$, $\mathbf{B}$, $\mathbf{H}$ durch zeitabhängige Versionen $\mathbf{F}_t$, $\mathbf{H}_t$ der partiellen Ableitungen von f nach $\mathbf{u}$ und h nach $\mathbf{x}$ (Jacobi-Matrizen)



Jacobi-Matrizen für F_t, H_t

Sei $F_t^{[i,j]}$ der (i,j) -te Eintrag in $\mathbf{F}_t$; seien $x^{[i]}, f^{[i]}$ i -te Einträge im Vektor $\mathbf{x}$ bzw der mehrdimensionalen Funktion f . (Entsprechend für $\mathbf{H}, h, u$)

$$\mathbf{F}_{t+1} = \left(F_{t+1}^{[i,j]} \right)_{i,j} = \frac{\partial f^{[i]}}{\partial x^{[j]}}(\mathbf{x}_t, \mathbf{u}_t) = \begin{pmatrix} \frac{\partial f^{[1]}}{\partial x^{[1]}} & \dots & \frac{\partial f^{[1]}}{\partial x^{[n]}} \\ \vdots & \ddots & \vdots \\ \frac{\partial f^{[n]}}{\partial x^{[1]}} & \dots & \frac{\partial f^{[n]}}{\partial x^{[n]}} \end{pmatrix}$$

$$\mathbf{H}_{t+1} = \left(H_{t+1}^{[i,j]} \right)_{i,j} = \frac{\partial h^{[i]}}{\partial x^{[j]}}(\underline{\mathbf{x}}_{t+1})$$



Eingabe: $\mathbf{x}_t$ aktueller Zustand, Σ_t aktuelles Fehlermodell, sowie ein Evidenz-Paar $(\mathbf{u}, \mathbf{z})$

Ausgabe: Die aktualisierten $\mathbf{x}_{t+1}, \Sigma_{t+1}$

1: *Prädiktion mit Hilfe der Aktion $\mathbf{u}$*

2: $\bar{\mathbf{x}}_{t+1} = f(\mathbf{x}_t, \mathbf{u})$

3: $\bar{\Sigma}_{t+1} = \mathbf{F}_{t+1} \Sigma_t \mathbf{F}_{t+1}^T + \Sigma_u$

4: *Korrektur mit Hilfe der Messung $\mathbf{z}$*

5: $\mathbf{K}_{t+1} = \bar{\Sigma}_{t+1} \mathbf{H}_{t+1}^T (\mathbf{H}_{t+1} \bar{\Sigma}_{t+1} \mathbf{H}_{t+1}^T + \Sigma_z)^{-1}$

6: $\mathbf{x}_{t+1} = \bar{\mathbf{x}}_{t+1} + \mathbf{K}_{t+1} (\mathbf{z} - h(\bar{\mathbf{x}}_{t+1}))$

// Zustand $\mathbf{x}_i \hat{=}$ Mittelwert μ_i

7: $\Sigma_{t+1} = (\mathbb{I} - \mathbf{K}_{t+1} \mathbf{H}_{t+1}) \bar{\Sigma}_{t+1}$

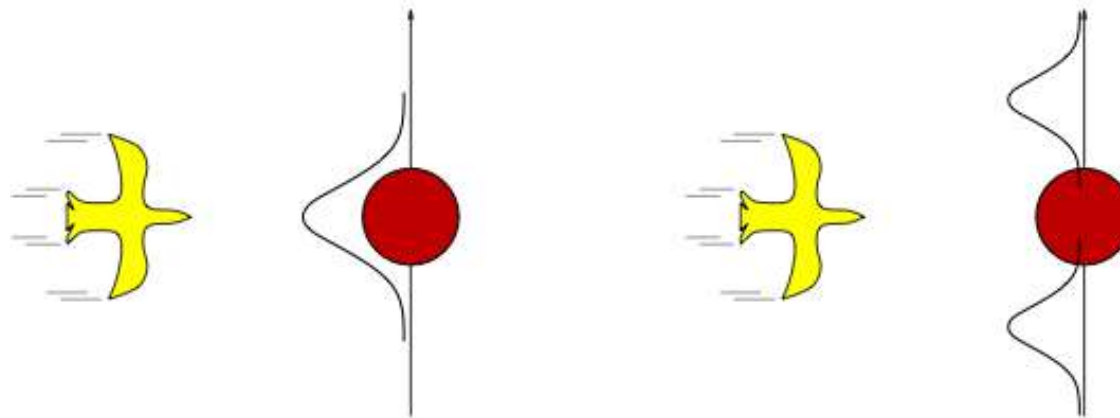
8: **return** $\mathbf{x}_{t+1}, \Sigma_{t+1}$

// Unverändert, falls $\mathbf{e} = \mathbf{u}$



Grenzen des Kalmanfilters

- ▶ Kalman-Filter werden praktisch auch angewandt, wenn Optimalitätsvoraussetzung nicht gegeben (z.B. korrelierte Fehlerquellen) ist
- ▶ Unimodale Verteilungen modellieren manche Aktionen nicht
- ▶ ➡ später arbeiten wir oft mit multimodalen Verteilungen (allg. Bayes-F.)



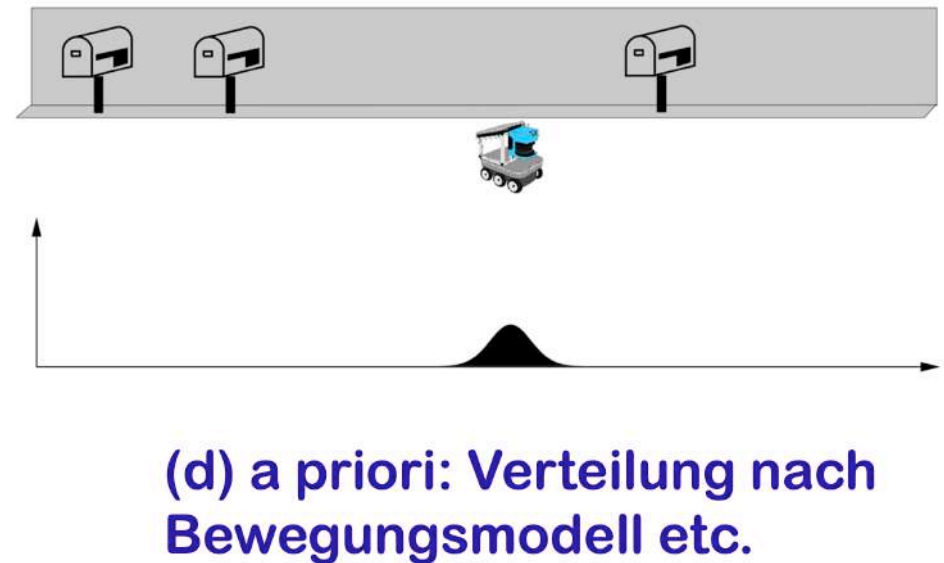
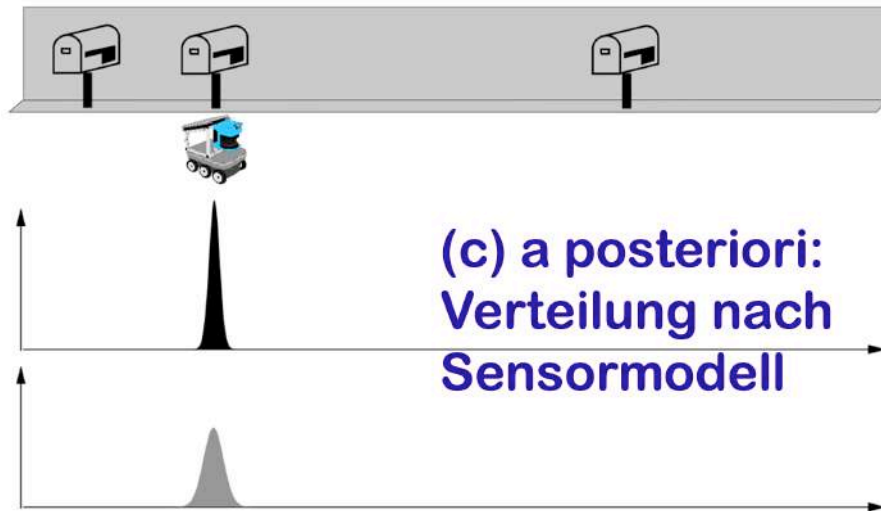
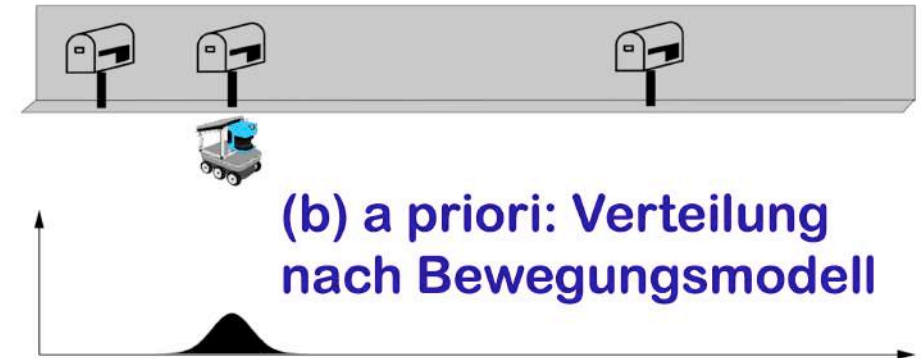
- ▶ In der Robotik sind Transitionen meist/oft nicht linear
- ▶ ➡ Erweiterter Kalman-Filter (EKF): Approximative Linearisierung eines nichtlinearen Transitions- und/oder Sensormodells (Jakobi-Matrizen d. nichtlinearen Funktionen)



- ▶ **Ziel:** Fusioniere 3D-Poseschätzung gemäß Vorwärtskinematik mit Korrekturwert aus Laserscan-Registrierung nach ICP
 - Kartenlokalisierung verwendet echten Scan an aktueller Pose und simulierten Scan nach Karte an geschätzter Pose
 - Derselbe Ansatz funktioniert ohne globale Karte mit zwei konsekutiven Scans
- ▶ **Ansatz:** Modelliere mit EKF (s. Kalman-Folien):
 - ▶ Zustandsvorhersage ($\underline{x}_{t+1}$) ergibt sich aus Poseschätzung
 - ▶ Messwert z ist Registrierungs-Korrekturvektor (umgerechnet ins Roboter-Bezugssystem)



Kalman-Lokalisierung - Schema





EKF-Lokalisierung (1)

- Erinnerung: $\mathbf{F}_t$ ersetzt $\mathbf{A}$ und $\mathbf{B}$; $\mathbf{H}$ wird zeit / zustands-abhängig

Eingabe: Akt. Zustand $\mathbf{x}_t$, akt. Fehlermodell Σ_t , $e \in \{u, z\}$ an $t+1$

Ausgabe: Aktualisierte $\mathbf{x}_{t+1}$, Σ_{t+1}

globale Variablen: $\mathbf{K}_i$, $\mathbf{x}_i$, Σ_i , **konstant:** Σ_u , Σ_z

if e ist Aktion u **then**

$$\mathbf{x}_{t+1} := f(\mathbf{x}_t, u)$$

$$\Sigma_{t+1} := \mathbf{F}_{t+1} \Sigma_t \mathbf{F}_{t+1}^T + \Sigma_u$$

else if e ist Messung z **then**

$$\mathbf{K}_{t+1} := \Sigma_{t+1} \mathbf{H}_{t+1}^T (\mathbf{H}_{t+1} \Sigma_{t+1} \mathbf{H}_{t+1}^T + \Sigma_z)^{-1}$$

$$\mathbf{x}_{t+1} := \mathbf{x}_{t+1} + \mathbf{K}_{t+1} (z - \mathbf{H}_{t+1} \mathbf{x}_{t+1})$$

$$\Sigma_{t+1} := (1 - \mathbf{K}_{t+1} \mathbf{H}_{t+1}) \Sigma_{t+1}$$

return($\mathbf{x}_{t+1}$, Σ_{t+1})

end if

Transitionsmodell (n.-lin.)

$$\mathbf{F}_{t+1} = \left(F_{t+1}^{[i,j]} \right)_{i,j} = \frac{\partial f^{[i]}}{\partial \mathbf{x}^{[j]}} (\mathbf{x}_t, \mathbf{u}_t)$$

Sensormodell (n.-lin.)

$$\mathbf{H}_{t+1} = \left(H_{t+1}^{[i,j]} \right)_{i,j} = \frac{\partial h^{[i]}}{\partial \mathbf{x}^{[j]}} (\mathbf{x}_{t+1})$$



EKF-Lokalisierung (2)

- ▶ Werde Fahr-Aktion u aus Pose $x_t = (x_t, z_t, \theta_t)^T$ ausgeführt. Vorwärtskinematik ergebe $u = (\Delta x_{odo}, \Delta z_{odo}, \Delta \theta_{odo})^T$.
- ▶ Zustandsschätzung $\underline{x}_{t+1}$ gemäß Vorwärtskinematik:

$$\begin{aligned}\bar{x}_{t+1} &= f(x_t, u) = x_t + R_{\theta_t} u \\ &= \begin{pmatrix} x_t \\ z_t \\ \theta_t \end{pmatrix} + \begin{pmatrix} \cos \theta_t & \sin \theta_t & 0 \\ -\sin \theta_t & \cos \theta_t & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \Delta x_{odo} \\ \Delta z_{odo} \\ \Delta \theta_{odo} \end{pmatrix} \\ &= \begin{pmatrix} x_t + \Delta x_{odo} \cos \theta_t + \Delta z_{odo} \sin \theta_t \\ z_t - \Delta x_{odo} \sin \theta_t + \Delta z_{odo} \cos \theta_t \\ \theta_t + \Delta \theta_{odo} \end{pmatrix} .\end{aligned}$$



- ▶ Scanregistrierung ergibt die Posedifferenz $z' = (\Delta x_{ICP}, \Delta z_{ICP}, \Delta \theta_{ICP})$. Lokale Messung muss in Referenzsystem umgerechnet werden
- ▶ Analog zu Vorwärtskinematik:

$$\begin{aligned} z &= x_t + R_{\theta_t} z' \\ &= \begin{pmatrix} x_t \\ z_t \\ \theta_t \end{pmatrix} + \begin{pmatrix} \cos \theta_t & \sin \theta_t & 0 \\ -\sin \theta_t & \cos \theta_t & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \Delta x_{ICP} \\ \Delta z_{ICP} \\ \Delta \theta_{ICP} \end{pmatrix} \\ &= \begin{pmatrix} x_t + \Delta x_{ICP} \cos \theta_t + \Delta z_{ICP} \sin \theta_t \\ z_t - \Delta x_{ICP} \sin \theta_t + \Delta z_{ICP} \cos \theta_t \\ \theta_t + \Delta \theta_{ICP} \end{pmatrix} . \end{aligned}$$



EKF-Lokalisierung (4)

- Funktion h dient beim EKF dazu, Messungen in Zustandsraum zu transformieren.
- Mess-Raum ist nach Umrechnungen ins globale Referenzsystem identisch mit Zustandsraum, daher ist h hier Identitätsfunktion:

$$f(\mathbf{x}_t, \mathbf{u}) = \begin{pmatrix} x_t + \Delta x_{\text{odo}} \cos \theta_t + \Delta z_{\text{odo}} \sin \theta_t \\ z_t - \Delta x_{\text{odo}} \sin \theta_t + \Delta z_{\text{odo}} \cos \theta_t \\ \theta_t + \Delta \theta_{\text{odo}} \end{pmatrix} \quad h(\bar{\mathbf{x}}_{t+1}) = \begin{pmatrix} \bar{x}_{t+1} \\ \bar{z}_{t+1} \\ \bar{\theta}_{t+1} \end{pmatrix}$$

$$\mathbf{F}_{t+1} = \left(\frac{\partial f^{[i]}}{\partial x^{[j]}}(\mathbf{x}_t, \mathbf{u}_t) \right)_{i,j} = \begin{pmatrix} 1 & 0 & -\Delta x_{\text{odo}} \sin \theta_t + \Delta z_{\text{odo}} \cos \theta_t \\ 0 & 1 & -\Delta x_{\text{odo}} \cos \theta_t - \Delta z_{\text{odo}} \sin \theta_t \\ 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{H}_{t+1} = \left(\frac{\partial h^{[i]}}{\partial \bar{x}^{[j]}}(\bar{\mathbf{x}}_{t+1}) \right)_{i,j} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} .$$



EKF-Lokalisierung (5)

- Kovarianzmatrizen für Aktionen und Messungen:

$$\Sigma_u = \begin{pmatrix} \sigma_{u,1}^2 & 0 & 0 \\ 0 & \sigma_{u,2}^2 & 0 \\ 0 & 0 & \sigma_{u,3}^2 \end{pmatrix} \quad \Sigma_z = \begin{pmatrix} \sigma_{z,1}^2 & 0 & 0 \\ 0 & \sigma_{z,2}^2 & 0 \\ 0 & 0 & \sigma_{z,3}^2 \end{pmatrix}$$

- Die σ repräsentieren die Unsicherheit des Kinematik- und des Sensormodells $\Rightarrow$ empirisch ermitteln oder schätzen:
 - Kinematikmodell okay für Translation, schwach für Rotation; also z.B.
 $\sigma_{u,1} = 0.1, \sigma_{u,2} = 0.2, \sigma_{u,3} = 0.8$
 - ICP-Registrierung okay bei Translation x,z; besser als Kinematik bei Rotation; also z.B.
 $\sigma_{z,1} = 0.3, \sigma_{z,2} = 0.3, \sigma_{z,3} = 0.4$



EKF - Rechenbeispiel

- ▶ ... zum Nachvollziehen gemäß Kalmanfilter (EKF)-Algorithmus. Hier nur 1 Bewegung, 1 Messung
- ▶ Details im Buch, Beispiel 5.3!

Sei $\mathbf{x}_0 = (0\text{cm}, 0\text{cm}, 0^\circ)^T$; $\Sigma_0 = \mathbf{0}$.

Odometrie gemäß Vorwärtskinematik ergebe

$\mathbf{u} = (0.25\text{cm}, 22.21\text{cm}, 0.65^\circ)^T$.

→ $\mathbf{x}_1 = f(\mathbf{x}_0, \mathbf{u}) = (0.25, 22.21, 0.65)^T$ (ist gleich Δ_{Odo} , da $\mathbf{x}_0 = (0, 0, 0)^T$)

→ $\bar{\Sigma}_1 = \begin{pmatrix} 0.01 & 0 & 0 \\ 0 & 0.04 & 0 \\ 0 & 0 & 0.64 \end{pmatrix} \quad \mathbf{K}_1 = \begin{pmatrix} 0.1 & 0 & 0 \\ 0 & 0.31 & 0 \\ 0 & 0 & 0.8 \end{pmatrix}$

Aus ICP-Registrierung folge $\mathbf{z} = (-1.98\text{cm}, 20.23\text{cm}, 1.2^\circ)$

→ Ergebnis $\mathbf{x}_1 = (0.029, 21.61, 1.09)^T$ mit Unsicherheit

$$\Sigma_1 = \begin{pmatrix} 0.009 & 0 & 0 \\ 0 & 0.028 & 0 \\ 0 & 0 & 0.128 \end{pmatrix}$$



Fazit Unimodale Lokalisierungsverfahren

- ▶ Bisher beschriebene Verfahren funktionieren unimodal, d.h. berechnen 1 neue a-posteriori-Pose (bzw. Hypothese)
- ▶ Grundlage ist praktisch immer die Poseschätzung gemäß Vorwärtskinematik (ggf. mit Verbesserungen, wie gehabt)
- ▶ Die meisten können als Modell-Datensatz Daten aus einer Roboterkarte oder aus einer früheren Sensormessung verwenden (absolute oder relative Lokalisierung)
- ▶ Unbekannte Startpose ist nicht behandelbar
- ▶ Geht die Pose einmal verloren, kann sie praktisch nicht mehr wiedergefunden werden: Kidnapped Robot Problem, aber auch (realistisch) Fahrt in merkmalsarmen Regionen



Gliederung

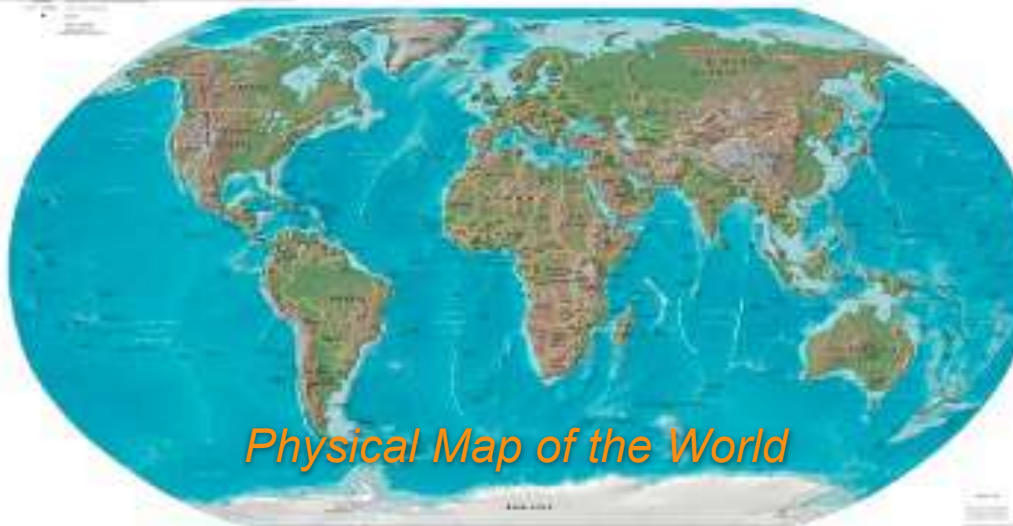
Lokalisierung

1. Grundlagen
2. Bayes- und Kalmanfilter
3. Inkrementelle Lokalisierung
- 4. Lokalisierung in Karten**
5. Mapping
6. Navigation
7. Ausblick



Karten

Es gibt nicht die Art Karte,
sondern endlos viele davon!



Physical Map of the World



Eine Roboterkarte ist eine explizite, persistente Repräsentation des Raums, die der Roboter für seine Steuerung nutzt.



Londoner U-Bahnnetz



Ebstorfer Weltkarte



Beispiele für 2D-Roboterkarten

- Roboterkarten ergeben sich aus dem Kreuzprodukt Modalität / Element

Element Modalität	Sensordaten	syntaktische Merkmale	semant. Merkmale Roboterbspl. Kap.8!
metrisch kontinuierlich			
metrisch diskret			
topologisch			



- ▶ **Metrisch kontinuierlich** (geometrische Belegungen im Raum)
 - Repräsentiere (x, z, θ) bzw. $(x, y, z, \theta_x, \theta_y, \theta_z)$ numerisch
 - Vorteil: Präzision, oft kompakte Repräsentation
 - Nachteil: Rechnen im Kontinuierlichen (z.B. W'dichten)
- ▶ **Metrisch diskret** (Raum „an sich“ und Belegungen)
 - repräsentiere (x, z) bzw. (x, y, z) diskret („Fliesen“, „Voxel“)
 - Vorteil: Uniformität, oft einfache Algorithmen
 - Nachteil: Diskretisierungsfehler
- ▶ **Topologisch** (i.d.R. diskret)
 - repräsentiere „Landmarken / Orte“ + „Wege dazwischen“
 - Vorteil: kompakte Repräsentation, einfache Algorithmen
 - Nachteil: Diskretisierungsfehler, viel Rauminfo. fehlt



- ▶ **Sensordaten** (Messwerte z.B. Laser-Punkte, Bilder)
 - Vorteil: Daten unverfälscht, vielfach interpretierbar
 - Nachteil: i.d.R. viel zu hohes Datenvolumen
- ▶ **Syntaktische Merkmale** („semantikfrei“ verarbeitet, z.B. Linien)
 - Vorteil: Komprimierung gegenüber Rohdaten,
 - Standard-Vorverarbeitung spart Zeit
 - Nachteil: ggf. fällt interessante Roh-Information weg
- ▶ **Semantische Merkmale** (z.B. Objektklassen)
 - Vorteil: Extrem hohe Komprimierung,
 - Speicherung in Termini der Nutzungskategorien
 - Erlauben Verknüpfung mit Wissensrepräsentation
 - Nachteil: Originaldaten nicht im Ansatz rekonstruierbar, wenn Nutzung sich ändert



► Lokalisierung

- Optimierte: Eindeutiges Matching von Sensordaten in Karte
- Kartenelemente: „Oberflächen“ (z.B. Linien/Flächen bei Abstandssensoren)
- Texturen / Bildfeatures bei Kameras

► Pfadplanung (hier Fahrpfade, allg. auch Greiftrajektorien)

- Optimierte: Rechenzeit für Planung kollisionsfreier Trajektorien von Start- nach Zielpose
- Kartenelemente: „Freiraum“-Regionen in 2D oder 3D

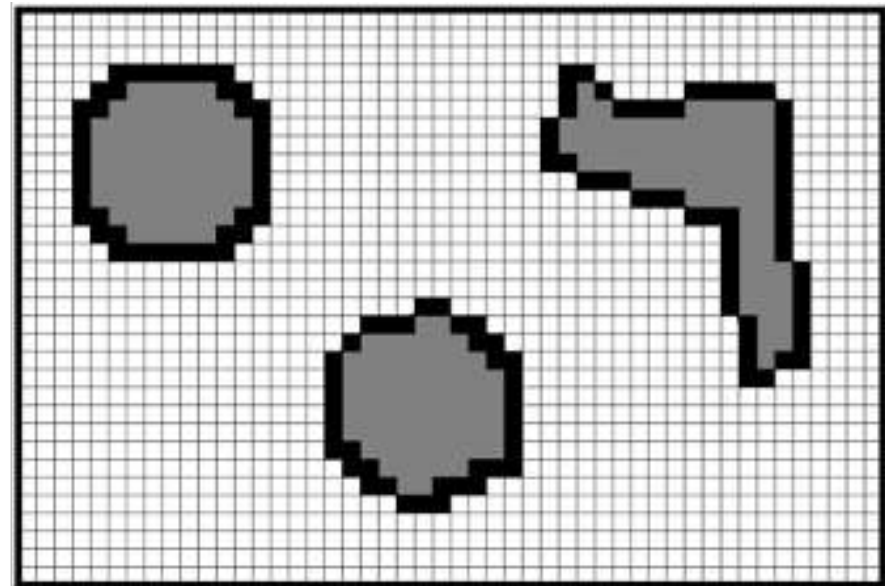
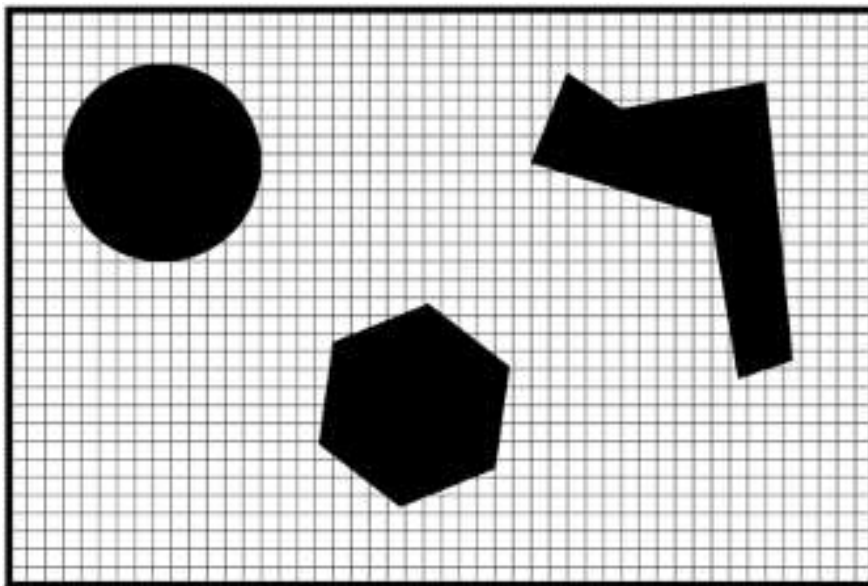
► Diverse Anwendungszwecke

- Optimierte: z.B. Präsentation für Nutzer, Archivierung, Auswertung
- Kartenelemente: Objekte/Kategorien der Anwendung
- (z.B. ATV-Schadensklassen bei Kanalinspektion)



Rasterkarten

- ▶ Lokalisierung und Pfadplanung
- ▶ Diskret Metrische Lokalisierung mit Entfernungs-Sensoren (Laser)
- ▶ Auch für Pfadplanung geeignet
- ▶ Standardverfahren in ROS
- ▶ Zerlege Raum in diskrete Elemente; markiere Belegtheit
- ▶ Vorteil: Sehr allgemeine, flexible Repräsentation
- ▶ Nachteil: Diskretisierungsfehler in Konturen, Pose, Freiraum

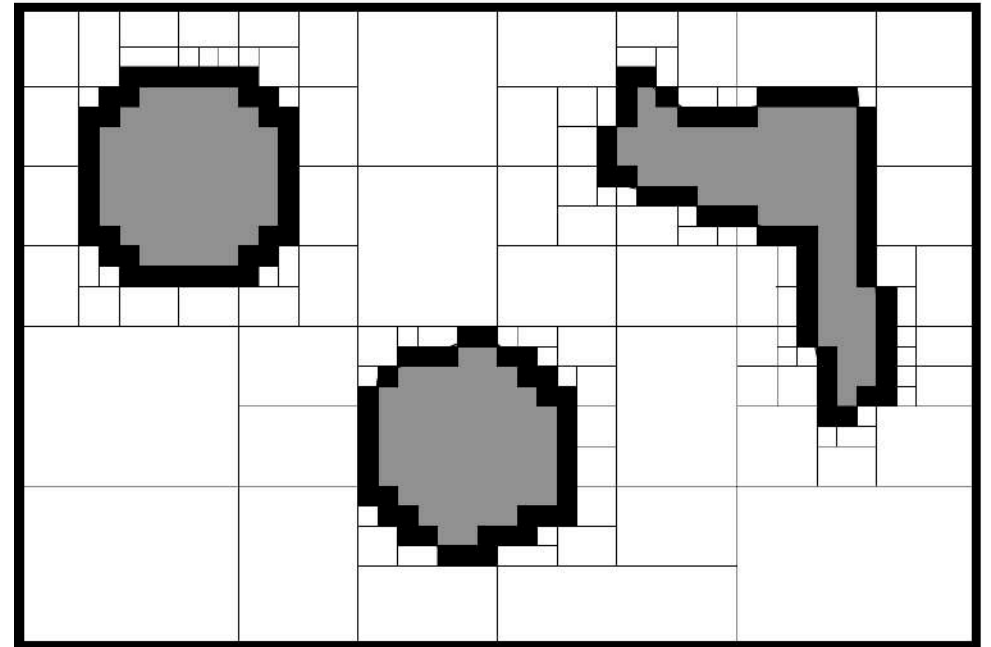
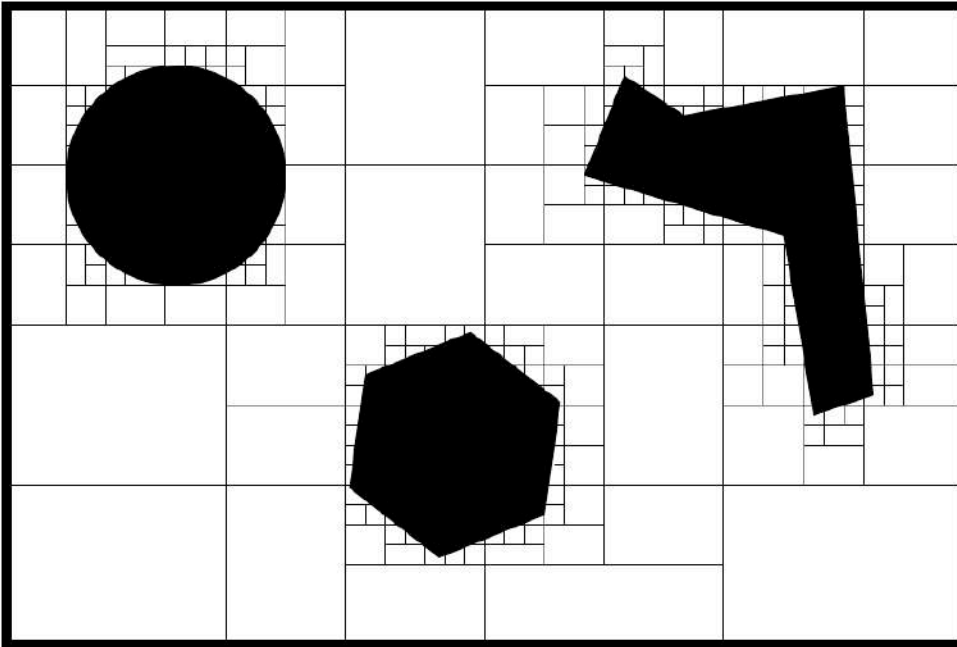




- ▶ Vorurteil (s. Siegwart / N.): „Rasterkarten kosten viel Speicher“
- ▶ Ist nur teilweise richtig: Quadrees (im 3D: Octrees) erlauben, spärlich besetzten Raum spärlich zu repräsentieren
- ▶ Repräsentations-Verfahren:
 - Starte mit (rechteckiger) Zelle, die Areal umfasst;
 - Enthält aktuelle Zelle belegte und freie Anteile, zerlege in 4 gleiche Teilzellen;
 - Stoppe Zerlegung, wenn Zelle komplett frei
 - Stoppe Zerlegung bei minimaler Auflösung (=Raster)
 - Markiere frei / belegt wie bei Rasterrepräsentation
- ▶ Octomap Package in ROS is de-facto Standard für viele Anwendungen!



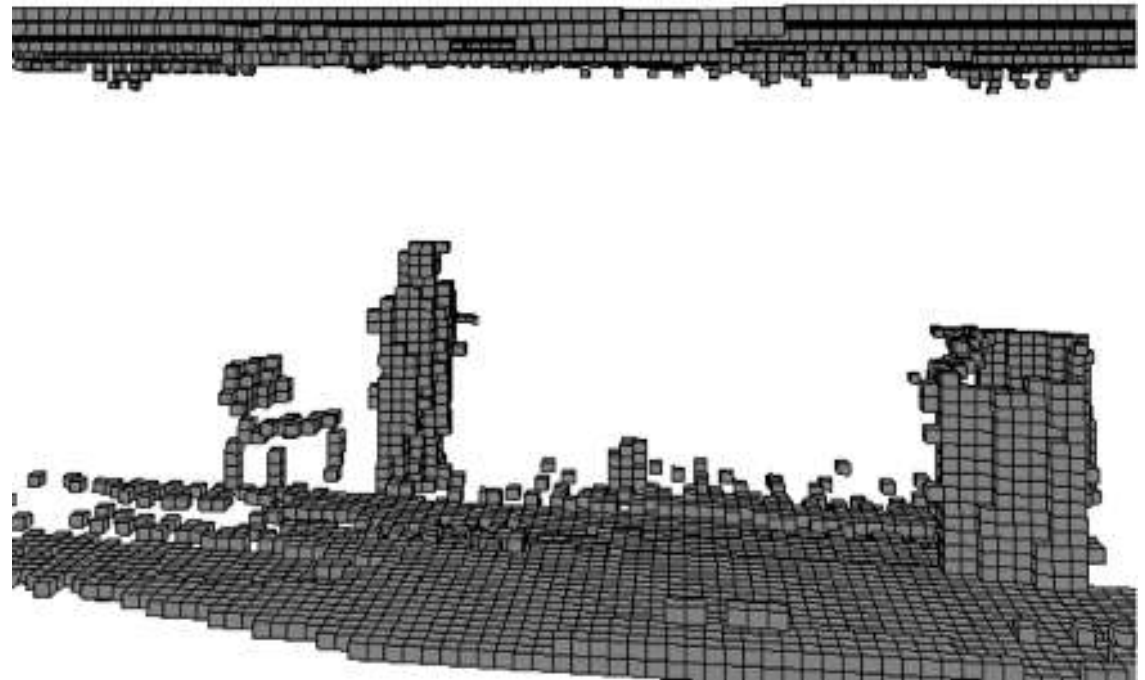
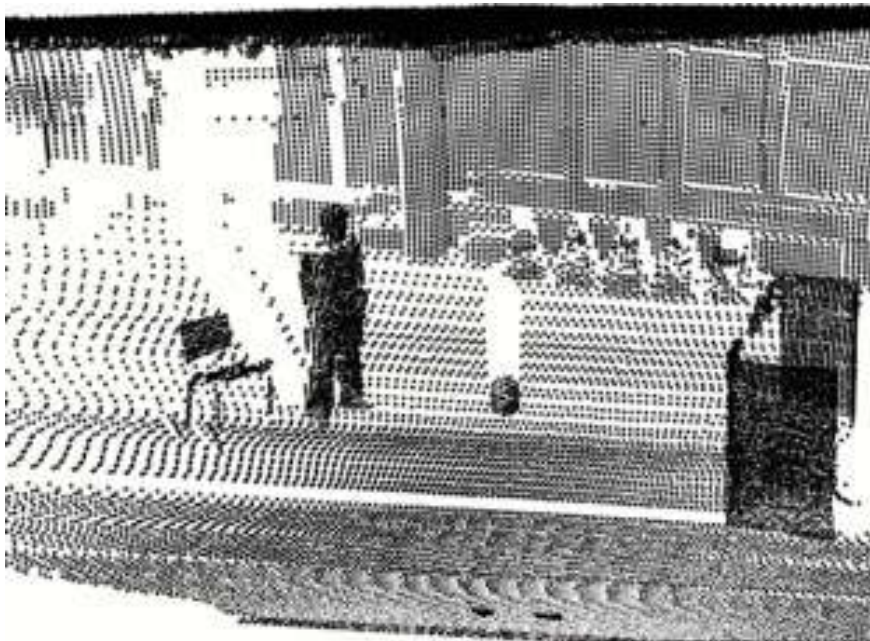
Quadtree-Rasterkarten





Octree-Karte

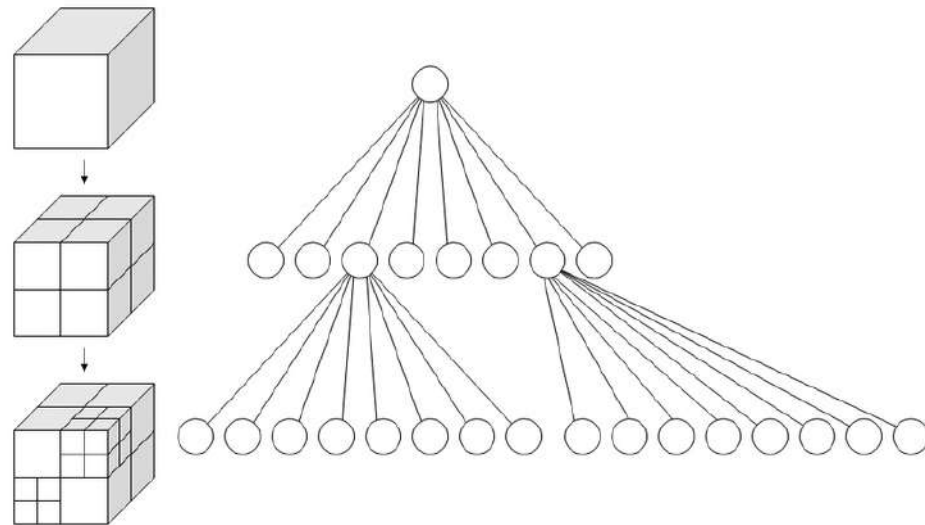
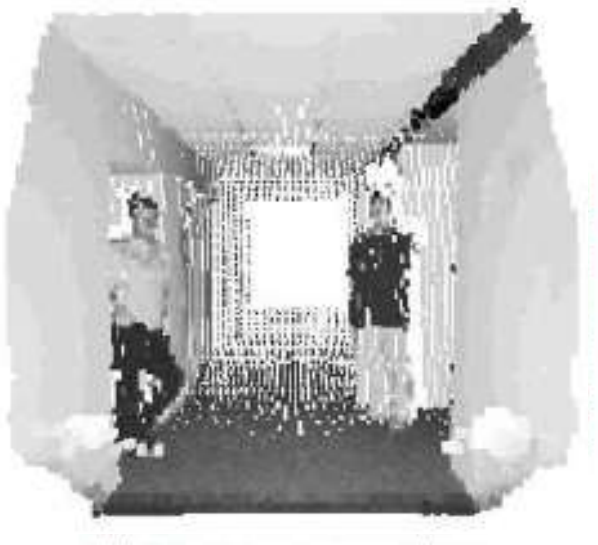
- ▶ Belegte Raumvolumina (deren Hüllen) sind immer in feinsten Auflösung!
- ▶ Voxelung des Freiraums nicht eingezeichnet





Octree - Erstellung

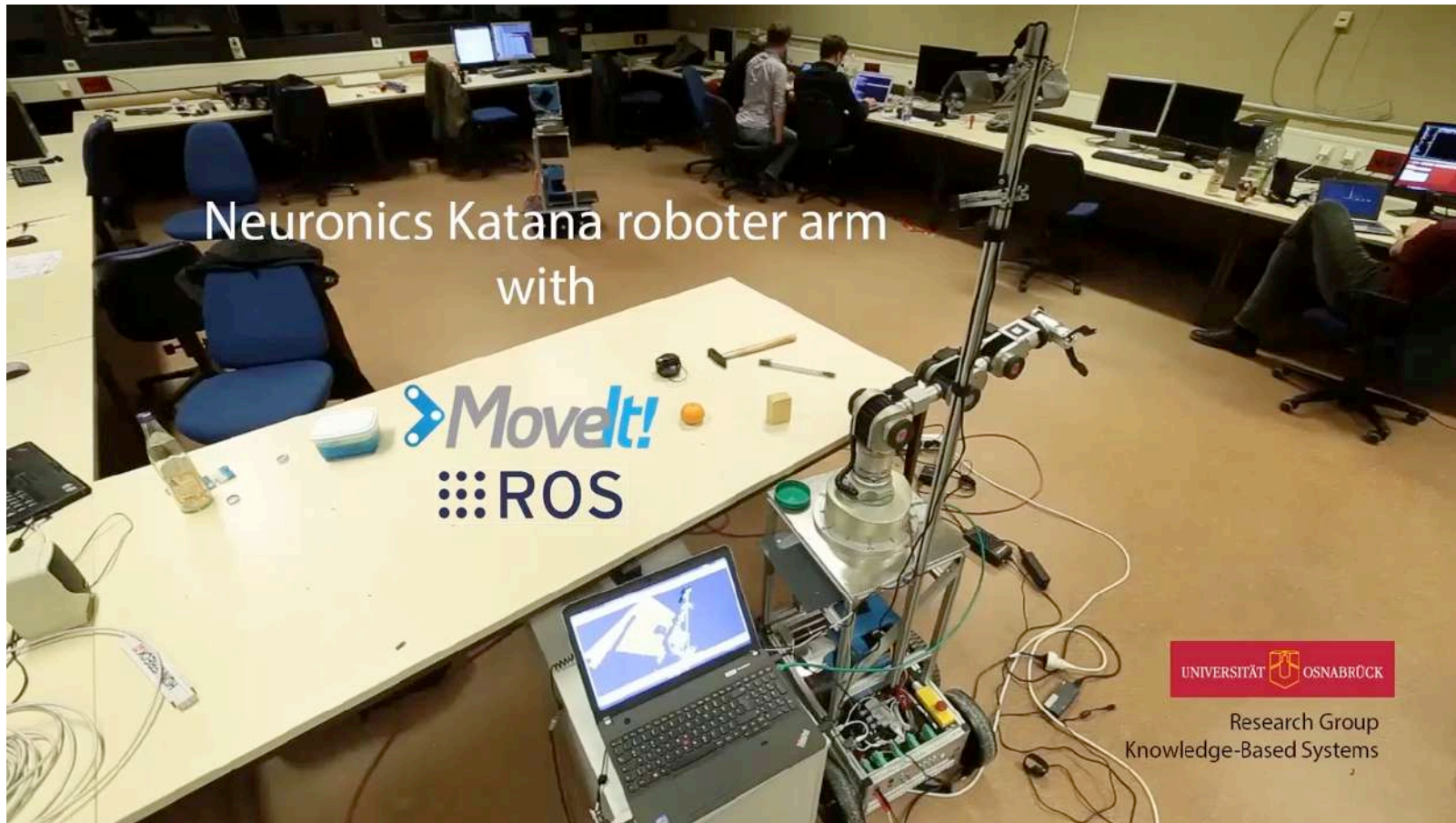
- Hier in 3D, 2D-Variante (Quadtree) analog...





Beispielanwendung für Octrees

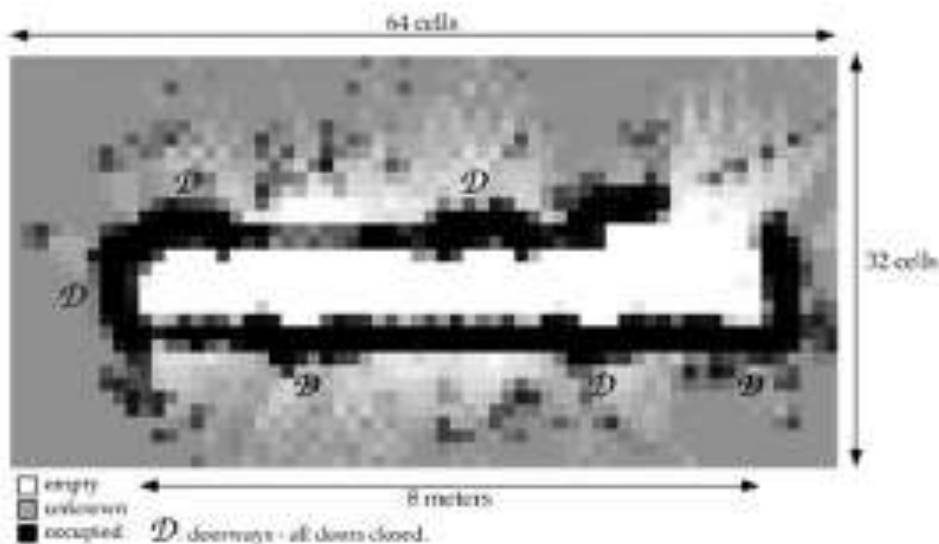
► Octomap und MoveIt in ROS



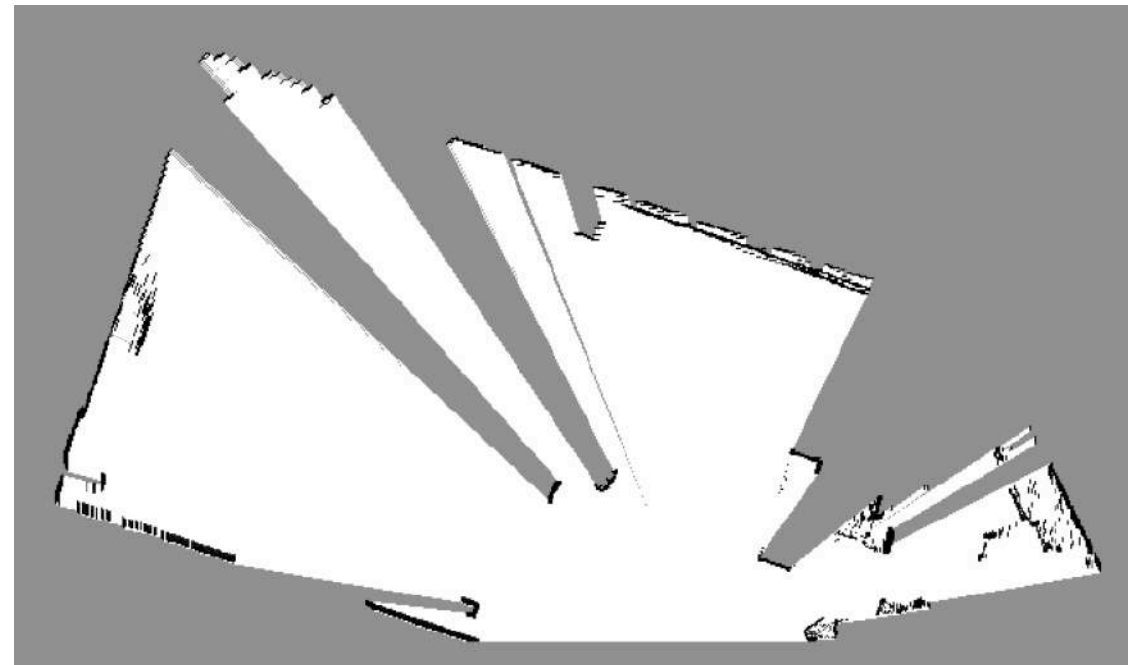


Belegtheitswahrscheinlichkeit

- Rasterkarten machen es leicht, Unsicherheit in Sensorwerten und transiente Information zu repräsentieren: Statt binärer Belegung speichere (aktuelle) Belegtheitswahrscheinlichkeit



Moravec/Elfes, 1985
Sonardaten, viele Messfehler
relativ grobe Auflösung



□ frei

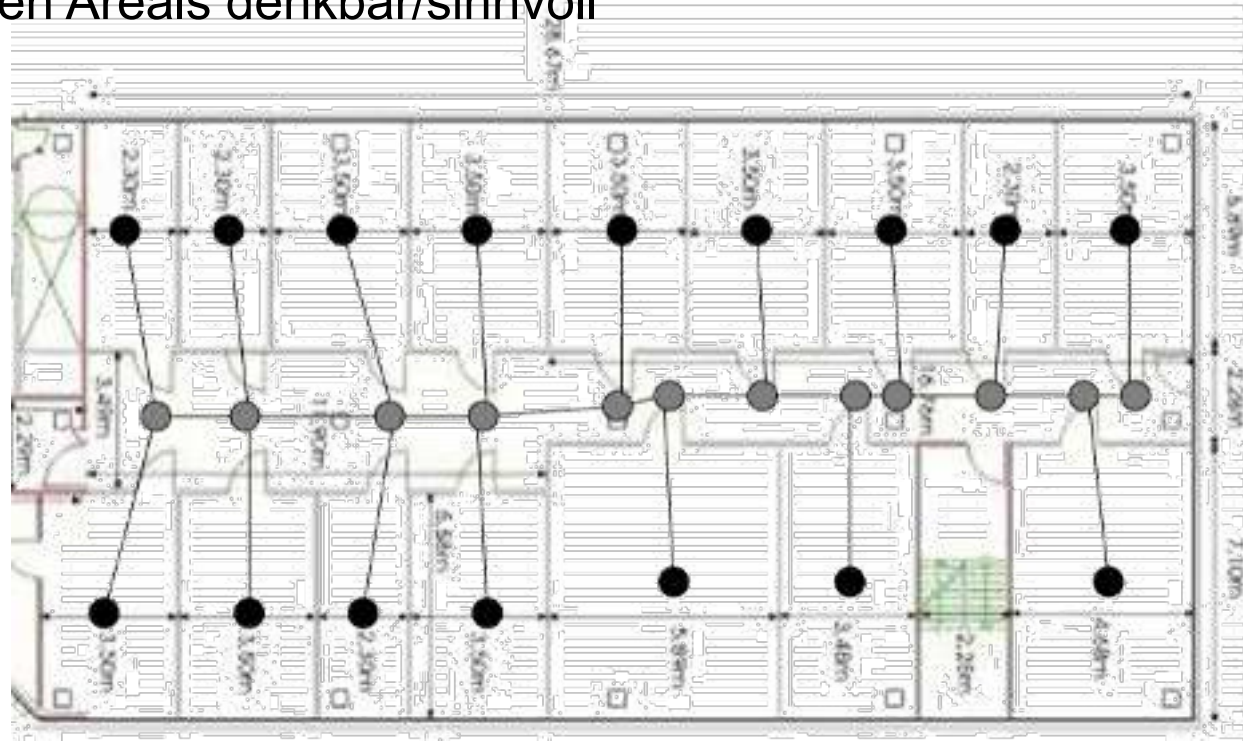
■ belegt

■ unbekannt



Topologische Karten zur Planung

- ▶ Abstrakte Zerlegung nach funktionalen/semantischen/... Kategorien (Abkehr von Repräsentation der Geometrie)
- ▶ Verbindungen zwischen direkt zugänglichen Knoten
- ▶ Mehrere Topologische Karten desselben Areals denkbar/sinnvoll
- ▶ Kommen in vielen Variationen vor
- ▶ Nah an „mental maps“





Semantische Karten

Eine semantische Karte ist eine Roboterkarte, die zusätzlich zu metrischen und geometrischen Informationen Zuordnungen von kartierten Strukturen und Objekten zu bekannten Konzepten und Relationen enthält. Weiterhin umfasst die semantische Karte statisches Wissen über diese Konzepte und Relationen, das in einer Wissensbasis abgelegt ist. Inferenzen sind möglich, welche die raumbezogene Information und das statische Wissen über Objekte in der semantischen Karte kombinieren

